

Вычислительные машины и системы. Архитектура фон Неймана

Лекция 1 · 04.09.2026

1 Организационные вопросы

Устное пояснение: Курс продолжает дисциплину «Теория формальных языков и грамматики», прочитанную в прошлом семестре: её теоретические основы — база процесса компиляции, то есть перевода программы с понятного человеку языка в машинные коды. В этом курсе тот же вопрос рассматривается со стороны процессора — как обрабатываются программы на машинном уровне. Знания низкоуровневого программирования полезны не только тем, кто работал с Ардуино: периодически приходится читать машинные коды или дизассемблированную программу.

Оценка выставляется по балльно-рейтинговой системе и учитывает посещение лекций, своевременность и качество сдачи лабораторных работ. Приём лабораторных работ не сводится к показу написанного кода — требуется объяснение и готовность сразу что-то дописать по ходу защиты: это единственный способ борьбы за самостоятельность выполнения работ.

2 Вычислительная машина и вычислительная система

2.1 Основные определения

Определение 1 (Вычислительная машина, ВМ) . Комплекс технических и программных средств, предназначенный для автоматизации подготовки и решения задач пользователей.

Определение 2 (Вычислительная система, ВС) . Совокупность взаимосвязанных и взаимодействующих процессоров или вычислительных машин, периферийного оборудования и программного обеспечения, предназначенную для подготовки и решения задач пользователей.

Устное пояснение: Прежде чем прийти к этой формулировке, в устном изложении перебирался ряд альтернативных определений ВМ: устройство, которое принимает данные, обрабатывает их по хранимой программе и генерирует результаты, обычно состоящее из блоков ввода, вывода, памяти, арифметики-логики и управления; функциональный блок, способный выполнять реальные вычисления (арифметические и логические операции) без обязательного участия человека; устройство, хранящее программу и необходимую для неё информацию, свободно перепрограмми-

руемое в соответствии с требованиями пользователя, способное менять последовательность действий в результате логических решений в процессе своей работы. Все эти формулировки верны — на слайде дано наиболее общее итоговое определение, которым курс и оперирует.

Предметом рассмотрения в курсе являются исключительно цифровые машины и системы — устройства, которые оперируют дискретными величинами.

2.2 Способы построения вычислительных машин

До недавнего времени были распространены примерно одинаково два способа построения ВМ — с непосредственными связями и на основе шины.

2.2.1 Архитектура с непосредственными связями

В классической фон-неймановской вычислительной машине взаимодействующие устройства (процессор, память, устройства ввода-вывода) имеют непосредственные связи между собой. Число линий в этих связях и их пропускная способность определяются видом информации, характером и интенсивностью обмена между устройствами.

Достоинство — возможность развязки узких мест: улучшать характеристики можно только для отдельных связей, а не для всей шины целиком, что экономически выгоднее. (Классическое узкое место фон-неймановских машин — канал передачи данных между процессором и памятью, и решается эта проблема достаточно сложно.)

Недостаток — такая архитектура плохо конфигурируется и реконфигурируется.

2.2.2 Шинная (магистральная) архитектура

Все устройства ВМ связаны между собой магистральной шиной — трактом для потоков команд, данных и управления, к которому подключены все устройства.

Достоинства — существенно упрощает реализацию ВМ и позволяет легко менять состав и конфигурацию, поэтому именно шинная архитектура получила широкое распространение в мини- и микроЭВМ.

Недостатки — в каждый момент времени передавать информацию по шине может только одно устройство: это и есть узкое место данной архитектуры. Основную нагрузку на шину создаёт обмен между процессором и памятью (нужно извлекать команды из памяти, выполнять их и записывать результат), поэтому операциям ввода-вывода остаются лишь остаточные ресурсы пропускной способности шины — даже при достаточно быстрой шине их на практике не хватает, особенно при обработке больших массивов данных или необходимости что-то визуализировать или передавать.

Устное пояснение: При сохранении фон-неймановской концепции последовательного выполнения команд шинная архитектура в чистом виде недостаточно эффективна, поэтому наиболее распространена архитектура с иерархией шин: помимо одной магистральной шины вводятся дополнительные быстрые шины для непосредственной связи между устройствами с интенсивным обменом (например, между процессором и кэш-памятью), либо однотипные устройства (например, устройства ввода-вывода) объединяются и подключаются к магистральной шине через отдельную дополнительную шину. Обе меры являются полумерами и лишь незначительно снижают нагрузку на общую шину.



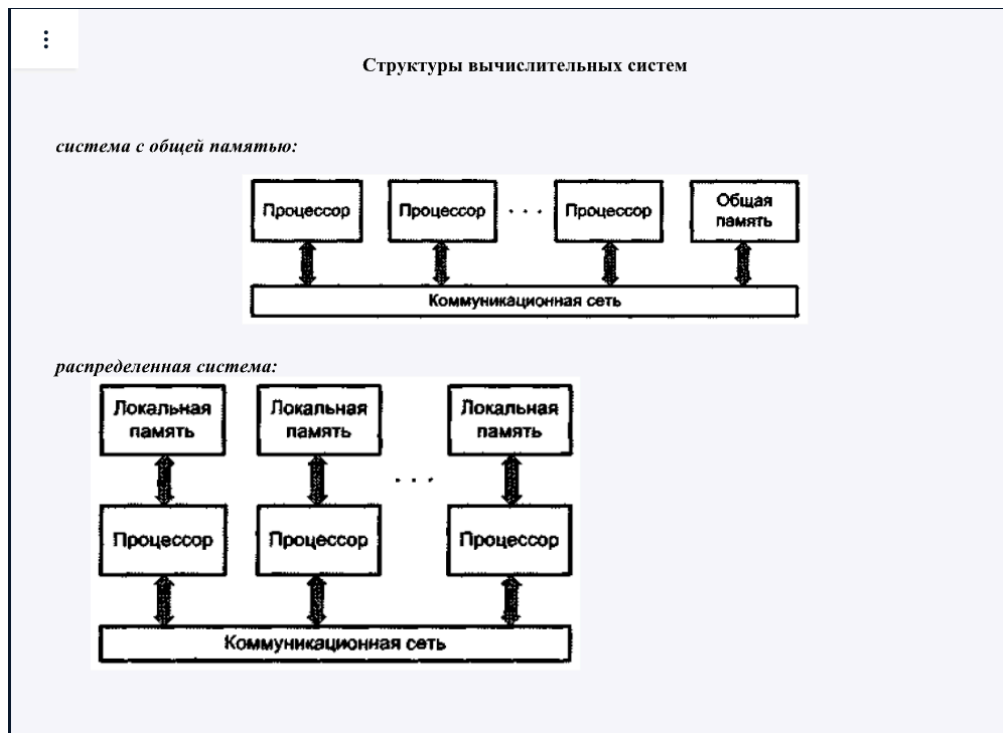
Слайд 1. Структуры вычислительных машин: с непосредственными связями и шинная

2.3 Структуры вычислительных систем

При объединении процессоров или вычислительных машин в систему используются два подхода.

Система с общей памятью. Имеется общая основная память, совместно используемая всеми процессорами системы; связь процессоров с памятью обеспечивается коммуникационной сетью, которая чаще всего реализуется в виде общей шины. По структуре и недостаткам такая система аналогична шинной архитектуре ВМ, но получает дополнительное достоинство: обмен между процессорами не требует дополнительных операций — он обеспечивается за счёт доступа к общим областям памяти.

Распределённая система. Общей памяти нет: у каждого процессора своя локальная память. Такие системы чаще всего объединяют отдельные вычислительные машины, а обмен информацией идёт через коммуникационную сеть посредством обмена сообщениями. Ограничения, свойственные системам с общей шиной, здесь снимаются, но появляются дополнительные издержки на пересылку сообщений.



Слайд 2. Структуры вычислительных систем: с общей памятью и распределённая

2.4 Основные показатели вычислительных машин

Использование конкретной ВМ имеет смысл, если её показатели соответствуют требованиям к реализации заданных алгоритмов, которые эта машина должна выполнять.

Определение 3 (Основные показатели ВМ) . Ёмкость памяти, быстродействие и производительность, стоимость и надёжность.

Определение 4 (Номинальное быстродействие) . Характеризует возможности ВМ при выполнении стандартной операции (обычно берут операцию сложения — самую короткую).

Определение 5 (Среднее быстродействие) . Характеризует скорость вычислений при выполнении эталонного алгоритма или некоторого класса алгоритмов.

Устное пояснение: Величина среднего быстродействия зависит не только от параметров вычислительной машины, но и от параметров самого алгоритма.

Определение 6 (Производительность) . Оценивается количеством эталонных алгоритмов, выполняемых ВМ в единицу времени.

Устное пояснение: Производительность — более универсальный показатель, чем среднее быстродействие.

Определение 7 (Эффективность) . Определяет степень соответствия ВМ своему назначению. Измеряется либо количеством затрат, необходимых для получения определённого результата, либо результатом, который получается при определённых затратах.

Определение 8 (Критерий эффективности) . Правило, служащее для сравнительной оценки качества вариантов ВМ.

Устное пояснение: Критерии эффективности (и правила предпочтения сравнимых вариантов) используются, когда нужно провести сравнительный анализ нескольких ВМ и решить, какую из них использовать для реализации конкретной задачи.

2.5 Архитектура и уровни детализации ВМ

Под архитектурой вычислительной машины обычно понимается её логическое построение — то, как машину видит программист. Термин впервые был применён фирмой IBM при разработке машин семейства IBM/360 для описания средств, которыми может пользоваться программист, составляя программу на уровне машинных команд: перечень и формат команд, формат представления данных, механизм ввода-вывода, способ адресации. Такую трактовку называют **узкой**, поскольку вопросы физического построения вычислительных средств — состав устройств, число регистров процессора, ёмкость памяти, наличие специального блока для обработки вещественных чисел, тактовая частота и т. п. — программисту недоступны. Круг таких вопросов принято определять понятием **организации** (структурной организации) вычислительной машины.

Устное пояснение: Такое деление на архитектуру в узком смысле и организацию ВМ не даёт курсу каких-либо преимуществ, поэтому здесь термин «архитектура» используется в более широком толковании — объединяя и архитектуру в узком смысле, и организацию вычислительной машины. Как программистам вам, конечно, ближе именно архитектура в узком смысле, но круг вопросов курса по большей части относится к компетенции системного архитектора.

Уровни детализации ВМ — как минимум четыре:

1. уровень чёрного ящика (известны вход и выход, внутреннее устройство не рассматривается);
2. уровень общей архитектуры;
3. уровень архитектуры центрального процессора;
4. уровень устройства управления.

Устное пояснение: Уровни (в) и (г) в этом курсе будут рассматриваться на достаточно общем, понятийном уровне — без разбора того, как это происходит на уровне реализации самих блоков.

3 Концепция машины с хранимой в памяти программой

3.1 Понятие алгоритма

Вычислительная машина — совокупность технических средств, служащих для автоматизированной обработки дискретных данных по заданному алгоритму, поэтому необходимо определиться с понятием алгоритма — одним из фундаментальных понятий математики и вычислительной техники.

Определение 9 (Алгоритм, ISO 2382/1-84) . Конечный набор предписаний, определяющий решение задачи посредством конечного количества операций.

Устное пояснение: Это не единственная формулировка. Также говорят, что алгоритм — это:

- способ преобразования информации, задаваемый с помощью конечной системы правил;
- совокупность правил, определяющих эффективную процедуру решения любой задачи из некоторого заданного класса задач;
- точно определённое правило действий, для которого задано указание, как и в какой последовательности его необходимо применять к исходным данным задачи, чтобы получить её решение.

Определение 10 (Основные свойства алгоритма) . Дискретность, определённость, массовость, результативность.

Устное пояснение:

- **Дискретность** — алгоритм работает с дискретной информацией (например, числовой или символьной), и сами действия тоже дискретны.
- **Определённость** — алгоритм включает все действия, которые должны быть сделаны, и ни одно из них не должно трактоваться двояко.
- **Массовость** — алгоритм применяется не к каким-то уникальным значениям, а к множеству исходных данных.
- **Результативность** — алгоритм должен получить результат за конечное число действий.

Именно эти свойства предопределяют возможность реализации алгоритма на вычислительной машине. Процесс, порождаемый алгоритмом, называется **вычислительным процессом**.

3.2 Архитектура фон Неймана

Определение 11 (Архитектура фон Неймана) . Классический принцип построения компьютеров, при котором программа и данные хранятся в одной и той же памяти и передаются по общим линиям связи.

Устное пояснение: Архитектура фон Неймана — модель организации компьютерной системы, предложенная в 1940-х годах американским математиком фон Нейманом и получившая его имя. Она описывает ключевые элементы, из которых должен состоять компьютер, и то, как

эти элементы взаимодействуют между собой; в той или иной мере ей следует большая часть современных компьютеров.

Компьютер состоит из процессора и оперативной памяти, соединённых шиной. Память используется как для хранения инструкций (команд), так и для хранения данных. Программа — набор команд, которые процессор выполняет последовательно и циклически: берёт из памяти команду, выполняет её, берёт следующую и так далее. Помимо процессора и оперативной памяти, которые являются ядром любой компьютерной системы, в неё входят устройства ввода и вывода информации — периферия.

Сущность фон-неймановской концепции сводится к четырём принципам:

1. принцип двоичного кодирования;
2. принцип программного управления;
3. принцип однородности памяти;
4. принцип адресности.

3.2.1 Принцип двоичного кодирования

Все цифровые микросхемы, в том числе компьютерные процессоры, работают только с двумя разрешёнными уровнями напряжения: логическому нулю (нулевому уровню) соответствует низкий уровень напряжения от 0 до 0,4 В, логической единице (единичному уровню) — высокий уровень напряжения от 2,5 до 5 В. Выбран этот принцип из-за простоты реализации: низкое напряжение — 0, высокое — 1. За такими цепочками нулей и единиц закрепилось название **машинные коды**; в виде последовательностей нулей и единиц информация записывается и на носители.

Устное пояснение: На этапе становления ЭВМ в прошлом веке предпринимались попытки ввести троичную логику (отечественная ЭВМ «Сетунь», 1959 год) и десятичную логику (американская ЭВМ Mark I, 1943 год). Такой способ хранения информации оказался сложнее, дороже и менее надёжным, чем на основе двоичной логики: всё, что осуществимо на машинах с любой другой логикой, можно реализовать на машинах двоичной логики с меньшими затратами по хранению информации.

Чтобы не записывать длинные цепочки нулей и единиц вручную, вместо двоичной системы счисления при записи используют и другие системы счисления — восьмеричную, десятичную, шестнадцатеричную. Система счисления числа обозначается буквой после него: **d** — десятичная, **b** — двоичная (бинарная), **o** — восьмеричная, **h** — шестнадцатеричная.

Код операции (КОп)	Адресная часть (АЧ)
какая операция должна быть выполнена	с какими данными и по какому адресу

Пример 1 (Перевод числа в разные системы счисления) .

$$200_d = 11001000_b = 310_o = C8_h$$

Перевод в двоичную систему делением на 2 (столбиком, снизу вверх по остаткам):

$$200/2 = 100 \text{ (ост. 0)}$$

$$100/2 = 50 \text{ (ост. 0)}$$

$$50/2 = 25 \text{ (ост. 0)}$$

$$25/2 = 12 \text{ (ост. 1)}$$

$$12/2 = 6 \text{ (ост. 0)}$$

$$6/2 = 3 \text{ (ост. 0)}$$

$$3/2 = 1 \text{ (ост. 1)}$$

$$1/2 = 0 \text{ (ост. 1)}$$

Результат (остатки снизу вверх): 11001000_b . Проверка:

$$11001000_b = 1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 128 + 64 + 8 = 200$$

Шестнадцатеричные цифры группируют двоичный код по 4 бита, например:

$$486_h = 10010000110_b$$

Такие длинные двоичные коды неудобны и для записи команд, поэтому исторически операторы первых ЭВМ программировали напрямую в машинных кодах, переключая специальные тумблеры либо пробивая коды на перфолентах или перфокартах: ЭВМ считывала эти двоичные коды и выполняла операции согласно им.

Устное пояснение: Образцы перфокарт и другой техники ЭВМ 1–4-го поколений можно увидеть в музее на пятом этаже лабораторного корпуса 5Л, собранном сотрудниками кафедры.

Для облегчения работы программистов был создан **язык ассемблера** — команды на нём записываются в более понятном человеку виде и легко переводятся в машинный код. Ассемблер называют низкоуровневым языком, поскольку он максимально приближён к машинному языку. Позже появились высокоуровневые языки (BASIC, Pascal, Fortran, C, C++) — они ещё более понятны человеку, но перевод программы на таком языке в машинный код существенно сложнее, а получаемый код обычно больше по размеру и медленнее ассемблерного.

*Устное пояснение: Программу-транслятор для высокоуровневых языков называют **компилятором**, а сам процесс — **компилированием**. Для языка ассемблера транслятор чаще называют просто **ассемблером**, а процесс перевода — **ассемблированием**. Обратный процесс — перевод программы из машинного кода (.exe, .com) в программу на языке ассемблера — называется **дизассемблированием**: он нужен, чтобы понять, что делает программа, если доступен только её машинный код.*

Каждая строка кода на ассемблере — это либо команда, либо директива (указание транслятору). Команды ассемблера легко запоминаются: обычно это 3–4 буквы от английского слова, обозначающего действие. Порядок в команде: код команды, затем приёмник, затем источник.

Пример 2 (Команда сложения) . Команда «сложить регистр `ax` с числом `33`»:

На языке ассемблера: `add ax, 33`

В машинном коде (шестнадцатеричном): `83h c0h 21h`

Байт (hex)	Двоичный код
83h	1000 0011
C0h	1100 0000
21h	0010 0001

Каждая машинная команда состоит из двух частей: **кода операции** — указания, какая операция должна быть выполнена, и **адресной части** — указания, с какими данными эта операция выполняется (сколько у команды операндов и где они расположены). Адресная часть может занимать разное число полей — от одного и больше.

⋮

Принцип двоичного кодирования

Код операции (Коп)	Адресная часть (Ач)
200d = 11001000b = 310o = C8h	
486h = 10010000110b	
200/2 = 100 (остаток 0)	
100/2 = 50 (остаток 0)	
50/2 = 25 (остаток 0)	
25/2 = 12 (остаток 1)	
12/2 = 6 (остаток 0)	
6/2 = 3 (остаток 0)	
3/2 = 1 (остаток 1)	
1/2 = 0 (остаток 1)	
Результат: 11001000b	
11001000b = 1×27 + 1×26 + 0×25 + 0×24 + 1×23 + 0×22 + 0×21 + 0×20 = 128 + 64 + 8 = 200	
add ax, 33	
83h C0h 21h	
100000111100000000100001	

Слайд 3. Принцип двоичного кодирования: перевод числа и кодирование команды `add ax, 33`

3.2.2 Принцип программного управления

Все вычисления, предусмотренные алгоритмом решения задачи, должны быть представлены в виде программы — последовательности управляющих слов, которые называют командами. Команда описывает (предписывает выполнить) некоторую операцию из набора операций, реализуемых данной вычислительной машиной.

Команды программы хранятся в последовательных ячейках памяти и выполняются в естественной последовательности — то есть в том порядке, в каком они лежат в памяти, — за исключением команд перехода, которые позволяют изменить порядок выполнения. Решение об изменении порядка выполнения команд принимается либо на основании результатов предшествующих вычислений (**условный переход**), либо специальной командой безусловного перехода.

3.2.3 Принцип однородности памяти

Команды и данные хранятся в одной и той же памяти и внешне (как последовательности нулей и единиц) никак не различимы. Это позволяет производить над командами те же операции, что и над числами, что открывает ряд возможностей: например, можно циклически изменять

адресную часть команды, чтобы обеспечить последовательное обращение к элементам массива данных — такой приём называется **модификацией команд**.

Устное пояснение: Модификация команд в современном программировании не приветствуется — слишком высока вероятность возникновения ошибок. А вот второе следствие принципа однородности — то, что команды одной программы могут быть получены как результат исполнения другой программы, — сейчас очень полезно и активно используется: именно на этой возможности основана **трансляция**, то есть перевод текста программы с языка высокого уровня в машинный код конкретной ЭВМ.

Согласно фон-неймановской архитектуре, откомпилированная (ассемблированная) программа перед непосредственным выполнением всегда помещается в память как последовательность инструкций, расположенных линейно друг за другом в двоичном виде. Процессор считывает команды из памяти в порядке их расположения и выполняет их; в зависимости от считанной команды процессор производит дальнейшие вычисления либо подаёт сигнал на какой-либо порт ввода-вывода (например, чтобы считать или записать информацию).

3.2.4 Принцип адресности

Основная память, в которой хранятся программы и данные, — это пронумерованные ячейки, хранящие информацию в виде последовательности байт; процессору в произвольный момент доступна любая ячейка памяти. Минимальная единица информации, с которой можно работать (считать, изменить, записать), — байт (8 бит): даже если нужно изменить значение всего одного бита, всё равно считывается целый байт, содержащий этот бит. Ячейки памяти не имеют названий (в отличие от регистров процессора) — они имеют уникальные числовые адреса, называемые физическими; нумерация начинается с нуля.

Память делится на:

- **оперативную** (RAM) — устройство с произвольным доступом, допускает и запись, и считывание;
- **постоянную** (ПЗУ, ROM) — используется только для чтения; в ПЗУ расположены BIOS и программа начальной загрузки компьютера (POST).

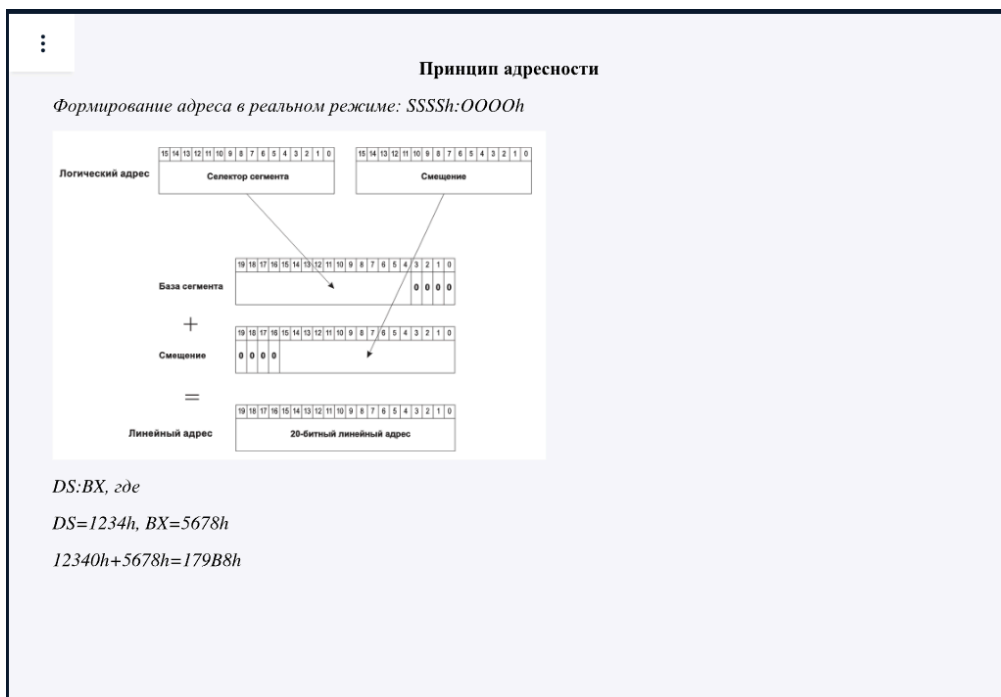
Устное пояснение: Оперативная и постоянная память находятся в едином пространстве адресов. Перед выполнением любая программа должна быть загружена в оперативную память — только после этого процессор может получить доступ к последовательности её команд и начать считывать и выполнять их одну за другой. Устройства постоянного хранения (жёсткие диски, CD, флеш-накопители) хранят файлы, которые сначала должны быть считаны и загружены в оперативную память — причём образ на носителе не всегда совпадает с образом, переносимым в память. Переносом информации между носителем и памятью занимается операционная система.

Стек. Загруженная в оперативную память программа всегда отводит под свои нужды отдельный участок памяти — стек. Данные в стек помещаются и извлекаются по принципу «последним вошёл — первым вышел» (доступ есть только к вершине стека). В системах Intel стек всегда расположен в больших адресах памяти (то есть в конце) и растёт по мере уменьшения адресов; между командами программы и стеком остаётся промежуток неиспользуемых адресов, что гарантирует, что стек не затрёт команды программы.

Сегментированная модель памяти. В реальном режиме — том, в котором ведётся курс, — процессор делит пространство памяти на сегменты по 64 КБ; такая модель памяти называется **сегментированной**. В защищённом режиме процессор предоставляет несколько моделей памяти, но чаще всего используется простейшая **плоская** модель, где память представляется одним сплошным массивом байт.

Устное пояснение: Деление памяти на сегменты по 64 КБ в реальном режиме — историческое ограничение: процессоры первого поколения Intel были 16-разрядными, поэтому максимальный адресуемый объём составлял ровно 64 КБ. Увеличить разрядность процессора тогда было невозможно, поэтому был введён специальный встроенный диспетчер памяти, для управления которым ввели регистры, хранящие адрес начала сегмента: *CS* (Code Segment) — адрес начала сегмента кода программы, *DS* (Data Segment) — адрес начала сегмента данных, *SS* (Stack Segment) — адрес начала сегмента стека. С появлением 32-разрядных процессоров размер сегментов тоже был увеличен.

Адрес в реальном режиме формируется из адреса сегмента и смещения внутри этого сегмента, где расположены нужные данные или команда. Базовый адрес сегмента умножают на 16 (то есть сдвигают на 4 бита) и складывают со смещением — получают физический адрес, по которому из оперативной памяти извлекаются данные или код (в зависимости от того, какой сегментный регистр использован). Так формируется 20-битный линейный адрес (шины тогда имели разрядность 20). Адрес, состоящий из адреса сегмента и смещения, называется **логическим** и записывается в виде `SSSh:0000h`.



Слайд 4. Принцип адресности: формирование адреса в реальном режиме

Пример 3 (Формирование физического адреса) .

$$DS = 1234_h, \quad BX = 5678_h$$

$$12340_h + 5678_h = 179B8_h$$

4 Дополнительные материалы

Устное пояснение: Рекомендованная литература по ассемблеру: Скляров, «Изучаем ассемблер за 7 дней»; книги Юрова, Калашиникова, Павлова; «Ассемблер для блондинок и блондинов» — как одна из самых доступных для восприятия. Курс изучает ассемблер для DOS, а не для Windows — из-за ограниченного числа лекций и лабораторных на изучение более сложного варианта не хватит времени.

Также рекомендован тематический YouTube-канал с короткими роликами по ассемблеру и переводу чисел между системами счисления¹.

¹Название канала в транскрипте распознано неразборчиво.